

Unit - 5 Working with Data in R

Working with data in R involves importing, exploring, cleaning, and manipulating data. R offers a variety of functions and packages to facilitate these processes. Commonly used packages include `dplyr`, `tidyverse`, and `data.table`.

Key Operations:

Importing Data: Using functions like `read.csv()` for CSV files, `read\_excel()` for Excel files, and `read.table()` for text files.

Exploring Data: Functions like `head()`, `summary()`, `str()`, and `View()` are used to get an overview of the data.

Manipulating Data: Functions like `filter()`, `select()`, `mutate()`, and `arrange()` from the `dplyr` package help in data manipulation.

Example:

```
library(dplyr)
```

Importing data

```
data <- read.csv("path/to/your/data.csv")
```

Viewing the first few rows

```
head(data)
```

Summary statistics

```
summary(data)
```

Selecting specific columns

```
selected_data <- data %>% select(Column1, Column2)
```

Filtering rows based on a condition

```
filtered_data <- data %>% filter(Column1 > 50)
```

2. Reordering and Reshaping Data Frames

Reordering and reshaping data frames are crucial for data analysis, allowing you to organize data in a format that is easy to analyze.

Reordering Data:

arrange(): Used to sort data based on one or more columns.

Example:

Sorting data by a column

```
sorted_data <- data %>% arrange(Column1)
```

Sorting in descending order

```
sorted_data_desc <- data %>% arrange(desc(Column1))
```

Reshaping Data:

pivot_longer(): Converts data from wide to long format.

pivot_wider(): Converts data from long to wide format.

Example:

```
library(tidyr)
```

Converting wide data to long format

```
long_data <- data %>% pivot_longer(cols = c(Column1, Column2), names_to = "Variable",  
values_to = "Value")
```

Converting long data to wide format

```
wide_data <- long_data %>% pivot_wider(names_from = Variable, values_from = Value)
```

Reordering and reshaping data frames are common tasks in data analysis. Below are examples of how to perform these tasks using R programming.

1. Reordering Data Frames

Reordering involves sorting data based on the values in one or more columns.

Example: Reordering a Data Frame by a Single Column

Suppose you have a data frame called `df`:

Sample data frame

```
df <- data.frame(  
  Name = c("John", "Alice", "Bob", "Carol"),  
  Age = c(28, 34, 22, 45),  
  Salary = c(50000, 60000, 45000, 70000)  
)
```

Display the original data frame

```
print(df)
```

Output:

	Name	Age	Salary
1	John	28	50000
2	Alice	34	60000
3	Bob	22	45000
4	Carol	45	70000

Reorder by `Age` (Ascending):

```
# Reorder by Age
```

```
df_sorted <- df[order(df$Age), ]
```

```
# Display the reordered data frame
```

```
print(df_sorted)
```

Output:

```
  Name Age Salary
3  Bob  22 45000
1  John 28 50000
2  Alice 34 60000
4  Carol 45 70000
```

Reorder by `Salary` (Descending):

```
# Reorder by Salary in descending order
```

```
df_sorted <- df[order(-df$Salary), ]
```

```
# Display the reordered data frame
```

```
print(df_sorted)
```

Output:

```
  Name Age Salary
4  Carol 45 70000
2  Alice 34 60000
1  John 28 50000
3  Bob  22 45000
```

2. Reshaping Data Frames

Reshaping involves transforming the structure of a data frame, often from wide format to long format or vice versa.

Example: Reshaping from Wide to Long Format

Suppose you have the following data frame in wide format:

Sample wide data frame

```
df_wide <- data.frame(  
  ID = c(1, 2, 3),  
  Year_2019 = c(50, 60, 70),  
  Year_2020 = c(55, 65, 75)  
)
```

Display the original wide data frame

```
print(df_wide)
```

Output:

	ID	Year_2019	Year_2020
1	1	50	55
2	2	60	65
3	3	70	75

Reshape to Long Format:

Using the `tidyr` package, you can convert the data from wide to long format.

Install and load the tidyr package

```
install.packages("tidyr")
```

```
library(tidyr)
```

Reshape from wide to long format

```
df_long <- gather(df_wide, key = "Year", value = "Score", Year_2019:Year_2020)
```

Display the reshaped long data frame

```
print(df_long)
```

Output:

	ID	Year	Score
1	1	Year_2019	50
2	2	Year_2019	60
3	3	Year_2019	70
4	1	Year_2020	55
5	2	Year_2020	65
6	3	Year_2020	75

Example: Reshaping from Long to Wide Format

You can also reshape data from long format back to wide format.

Reshape from long to wide format

```
df_wide_again <- spread(df_long, key = "Year", value = "Score")
```

Display the reshaped wide data frame

```
print(df_wide_again)
```

Output:

ID	Year_2019	Year_2020
----	-----------	-----------

1	1	50	55
2	2	60	65
3	3	70	75

3. Merging and Joining Data Frames

Merging and joining data frames is essential when working with multiple datasets. R provides several functions to merge data based on common columns or keys.

Key Functions:

merge(): Base R function for merging data frames.

left_join(), right_join(), inner_join(), full_join(): Functions from `dplyr` for joining data frames.

Example:

Base R merge

```
merged_data <- merge(data1, data2, by = "CommonColumn")
```

dplyr left join

```
joined_data <- data1 %>% left_join(data2, by = "CommonColumn")
```

dplyr inner join

```
inner_joined_data <- data1 %>% inner_join(data2, by = "CommonColumn")
```

Load the dplyr package

```
library(dplyr)
```

Create the first data frame (df1)

```
df1 <- data.frame(
```

```
ID = c(1, 2, 3, 4),  
Name = c("Alice", "Bob", "Charlie", "David"),  
Age = c(24, 27, 22, 32)  
)
```

Create the second data frame (df2)

```
df2 <- data.frame(  
  ID = c(3, 4, 5, 6),  
  Department = c("HR", "Finance", "IT", "Operations"),  
  Salary = c(50000, 60000, 55000, 65000)  
)
```

View the data frames

```
print(df1)
```

```
print(df2)
```

1. left_join()

`left_join()` returns all rows from the left data frame (df1), and the matched rows from the right data frame (df2). If there is no match, NA values are returned for columns from the right data frame.

Perform a left join

```
left_join_df <- left_join(df1, df2, by = "ID")  
print(left_join_df)
```

Output:

	ID	Name	Age	Department	Salary
1	1	Alice	24	<NA>	NA
2	2	Bob	27	<NA>	NA
3	3	Charlie	22	HR	50000
4	4	David	32	Finance	60000

2. right_join()

right_join() returns all rows from the right data frame (df2), and the matched rows from the left data frame (df1). If there is no match, NA values are returned for columns from the left data frame.

Perform a right join

```
right_join_df <- right_join(df1, df2, by = "ID")  
print(right_join_df)
```

Output:

	ID	Name	Age	Department	Salary
1	3	Charlie	22	HR	50000
2	4	David	32	Finance	60000
3	5	<NA>	NA	IT	55000
4	6	<NA>	NA	Operations	65000

3. inner_join()

inner_join() returns only the rows that have matching values in both data frames.

Perform an inner join

```
inner_join_df <- inner_join(df1, df2, by = "ID")  
print(inner_join_df)
```

Output:

Copy code

	ID	Name	Age	Department	Salary
1	3	Charlie	22	HR	50000
2	4	David	32	Finance	60000

4. full_join()

full_join() returns all rows when there is a match in either the left or right data frame. Non-matching rows will have NA values.

Perform a full join

```
full_join_df <- full_join(df1, df2, by = "ID")  
print(full_join_df)
```

Output:

	ID	Name	Age	Department	Salary
1	1	Alice	24	<NA>	NA
2	2	Bob	27	<NA>	NA
3	3	Charlie	22	HR	50000
4	4	David	32	Finance	60000
5	5	<NA>	NA	IT	55000

4. Calculating Summary Statistics

Summary statistics provide insights into the distribution and central tendency of your data. R offers several functions to calculate these statistics.

Key Functions:

mean(): Calculates the mean.

median(): Calculates the median.

mode(): Can be calculated using a custom function.

sd(): Calculates the standard deviation.

Example:

Calculating mean

```
mean_value <- mean(data$Column1, na.rm = TRUE)
```

Calculating median

```
median_value <- median(data$Column1, na.rm = TRUE)
```

Calculating mode

```
get_mode <- function(v) {  
  uniqv <- unique(v)  
  uniqv[which.max(tabulate(match(v, uniqv)))]  
}  
  
mode_value <- get_mode(data$Column1)
```

Calculating standard deviation

```
sd_value <- sd(data$Column1, na.rm = TRUE)
```

5. Generating Frequency Tables and Cross-Tabulations

Frequency tables and cross-tabulations help summarize categorical data, showing the distribution of data across different categories.

Key Functions:

table(): Generates a frequency table.

prop.table(): Generates a table of proportions.

xtabs(): Creates cross-tabulations.

Example:

Frequency table

```
freq_table <- table(data$CategoricalColumn)
```

Proportion table

```
prop_table <- prop.table(freq_table)
```

Cross-tabulation

```
cross_tab <- xtabs(~ Column1 + Column2, data = data)
```

Here are examples of how to generate frequency tables and cross-tabulations using R programming.

1. Generating Frequency Tables

Frequency tables are useful for summarizing the distribution of a categorical variable. You can use the `table()` function in R to create a frequency table.

Example:

Let's say you have a vector representing the favorite fruit of a group of people:

Create a vector of favorite fruits

```
favorite_fruits <- c("Apple", "Banana", "Apple", "Orange", "Banana", "Apple", "Mango",  
"Orange", "Mango", "Apple")
```

Generate a frequency table

```
fruit_freq_table <- table(favorite_fruits)  
  
print(fruit_freq_table)
```

Output:

favorite_fruits

Apple	Banana	Mango	Orange
-------	--------	-------	--------

4	2	2	2
---	---	---	---

This output shows the frequency of each fruit in the `favorite\_fruits` vector.

2. Generating Cross-Tabulations

Cross-tabulations (or contingency tables) summarize the relationship between two categorical variables. You can use the `table()` function to create a cross-tabulation.

Example:

Let's consider a scenario where you have two vectors, one representing the gender of a person and the other representing their preferred fruit:

Create a vector of genders

```
genders <- c("Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female",  
"Male", "Female")
```

```
# Create a vector of favorite fruits
```

```
favorite_fruits <- c("Apple", "Banana", "Apple", "Orange", "Banana", "Apple", "Mango",  
"Orange", "Mango", "Apple")
```

```
# Generate a cross-tabulation
```

```
gender_fruit_table <- table(genders, favorite_fruits)  
print(gender_fruit_table)
```

Output:

```
favorite_fruits  
genders Apple Banana Mango Orange  
Female    2     1         1         2  
Male      2     1         1         0
```

This output shows how many males and females prefer each type of fruit.

3. Cross-Tabulations with `addmargins()`

To include row and column totals in your cross-tabulation, you can use the `addmargins()` function.

Example:

```
# Add row and column totals
```

```
gender_fruit_table_with_margins <- addmargins(gender_fruit_table)  
print(gender_fruit_table_with_margins)
```

Output:

	favorite_fruits				
genders	Apple	Banana	Mango	Orange	Sum
Female	2	1	1	2	6
Male	2	1	1	0	4
Sum	4	2	2	2	10

This table includes the total counts for each row and column.

4. Using `ftable()` for Multi-Dimensional Cross-Tabulations

If you have more than two categorical variables, you can use the `ftable()` function to create a more readable flat contingency table.

Example:

Create a vector of age groups

```
age_groups <- c("Youth", "Adult", "Youth", "Adult", "Youth", "Adult", "Youth", "Adult",
               "Youth", "Adult")
```

Generate a multi-dimensional cross-tabulation

```
multi_cross_tab <- ftable(genders, age_groups, favorite_fruits)
print(multi_cross_tab)
```

Output:

		favorite_fruits			
genders	age_groups	Apple	Banana	Mango	Orange
Female	Adult	0	1	1	2
	Youth	2	0	0	0
Male	Adult	0	0	1	0
	Youth	2	1	0	0

This output shows the distribution of favorite fruits across different gender and age groups.

6. Understanding Measures of Central Tendency and Dispersion

Central tendency refers to the center of a data distribution, while dispersion measures the spread of the data.

Central Tendency:

Mean: The average of the data.

Median: The middle value when data is sorted.

Mode: The most frequent value in the data.

Dispersion:

Range: The difference between the maximum and minimum values.

Variance: The average of the squared differences from the mean.

Standard Deviation: The square root of the variance.

Example:

Calculating range

```
range_value <- range(data$Column1)
```

Calculating variance

```
variance_value <- var(data$Column1, na.rm = TRUE)
```

Calculating standard deviation

```
sd_value <- sd(data$Column1, na.rm = TRUE)
```

Here are examples to understand measures of central tendency (mean, median, mode) and measures of dispersion (range, variance, standard deviation) using R programming.

1. Measures of Central Tendency

a. Mean

The mean is the average of all the data points.

Example:

Create a vector of data

```
data <- c(10, 20, 30, 40, 50)
```

Calculate the mean

```
mean_value <- mean(data)
```

```
print(mean_value)
```

Output:

```
[1] 30
```

b. Median

The median is the middle value in a dataset when arranged in ascending or descending order.

Example:

Calculate the median

```
median_value <- median(data)
```

```
print(median_value)
```

Output:

```
[1] 30
```


c. Mode

The mode is the value that appears most frequently in the dataset. R does not have a built-in function for mode, so we need to create a custom function.

Example:

Custom function to calculate the mode

```
get_mode <- function(v) {  
  uniq_v <- unique(v)  
  uniq_v[which.max(tabulate(match(v, uniq_v)))]  
}
```

Calculate the mode

```
mode_value <- get_mode(data)  
print(mode_value)
```

Output:

```
[1] 10 20 30 40 50
```

In this example, all values appear only once, so each value can be considered a mode.

2. Measures of Dispersion

a. Range

The range is the difference between the maximum and minimum values.

Example:

Calculate the range

```
range_value <- range(data)
```

```
print(range_value)
```

Output:

```
[1] 10 50
```

The range of the data is from 10 to 50.

b. Variance

Variance measures the spread of the data points from the mean.

Example:

```
# Calculate the variance
```

```
variance_value <- var(data)
```

```
print(variance_value)
```

Output:

```
[1] 250
```

c. Standard Deviation

Standard deviation is the square root of variance and provides the spread of data points in the same units as the data.

Example:

```
# Calculate the standard deviation
```

```
std_dev_value <- sd(data)
```

```
print(std_dev_value)
```

Output:

```
[1] 15.81139
```

Summary Example

Here's a summary example that calculates all these measures at once:

Data vector

```
data <- c(10, 20, 30, 40, 50)
```

Central Tendency

```
mean_value <- mean(data)
```

```
median_value <- median(data)
```

```
mode_value <- get_mode(data)
```

Dispersion

```
range_value <- range(data)
```

```
variance_value <- var(data)
```

```
std_dev_value <- sd(data)
```

Print results

```
cat("Mean:", mean_value, "\n")
```

```
cat("Median:", median_value, "\n")
```

```
cat("Mode:", mode_value, "\n")
```

```
cat("Range:", range_value, "\n")
```

```
cat("Variance:", variance_value, "\n")
```

```
cat("Standard Deviation:", std_dev_value, "\n")
```

Output:

```
Mean: 30
```

Median: 30

Mode: 10 20 30 40 50

Range: 10 50

Variance: 250

Standard Deviation: 15.81139

7. Exploring Data Distributions Graphically (Bell Curve)

Graphical exploration of data distributions helps in understanding the shape and spread of the data. The bell curve, or normal distribution, is a common way to visualize continuous data.

Key Functions and Packages:

hist(): Creates a histogram.

density(): Estimates the density of the data.

ggplot2: A powerful package for creating complex visualizations.

Example:

```
library(ggplot2)
```

```
# Histogram with a bell curve
```

```
ggplot(data, aes(x = Column1)) +
```

```
  geom_histogram(aes(y = ..density..), bins = 30, color = "black", fill = "blue") +
```

```
  geom_density(color = "red", size = 1) +
```

```
  ggtitle("Bell Curve of Column1") +
```

```
  xlab("Column1") +
```

```
  ylab("Density")
```

Exploring data distributions graphically, particularly using a bell curve, involves plotting the normal distribution of the data. Here's how you can do this in R:

Example: Plotting a Bell Curve

1. Generate Normally Distributed Data

To plot a bell curve, we'll first create a dataset that follows a normal distribution.

Set seed for reproducibility

```
set.seed(123)
```

Generate a normally distributed dataset with mean = 50 and standard deviation = 10

```
data <- rnorm(1000, mean = 50, sd = 10)
```

2. Plotting the Histogram with a Bell Curve

We can now plot a histogram of the data and overlay a bell curve to visualize the normal distribution.

Create a histogram

```
hist(data,  
      breaks = 30,  
      probability = TRUE, # To show probability density rather than frequency  
      main = "Histogram with Normal Curve",  
      xlab = "Data",  
      col = "lightblue",  
      border = "black")
```

Add a bell curve (normal distribution curve) to the histogram

```
curve(dnorm(x, mean = mean(data), sd = sd(data)),  
      col = "darkblue",  
      lwd = 2,  
      add = TRUE)
```

Explanation:

- ✓ ``rnorm(1000, mean = 50, sd = 10)``: Generates 1,000 random numbers from a normal distribution with a mean of 50 and a standard deviation of 10.
- ✓ ``hist()``: Creates a histogram of the data. The ``probability = TRUE`` argument scales the y-axis to represent probability density instead of frequency.
- ✓ ``curve()``: Adds a normal distribution curve on top of the histogram. The ``dnorm()`` function generates the density values for the normal distribution curve using the mean and standard deviation of the data.

3. Example of a Bell Curve with Density Plot

An alternative to overlaying a bell curve on a histogram is to use a density plot, which provides a smooth estimate of the distribution.

Create a density plot

```
plot(density(data),  
     main = "Density Plot with Bell Curve",  
     xlab = "Data",  
     col = "darkblue",  
     lwd = 2)
```

Add a bell curve to the density plot

```
curve(dnorm(x, mean = mean(data), sd = sd(data)),
```

```
col = "red",  
lwd = 2,  
add = TRUE,  
lty = 2)
```

Explanation:

- ✓ ``density(data)``: Computes the density estimate of the data.
- ✓ The ``curve()`` function again overlays the normal distribution curve, but this time on the density plot, and uses a dashed line (``lty = 2``) to differentiate it from the density estimate.

SHORT QUESTIONS AND ANSWERS

Here are 20 short questions and answers related to the points you've provided:

1. Working with Data in R

Q1: How do you create a data frame in R?

A1: Use the `data.frame()` function to create a data frame by passing vectors as arguments.

Q2: What function is used to view the first few rows of a data frame?

A2: The `head()` function is used to view the first few rows of a data frame.

Q3: How can you access a specific column in a data frame?

A3: Use the `$` operator or double square brackets `[[ ]]` to access a specific column, e.g., `df$columnName` or `df[[columnName]]`.

2. Reordering and Reshaping Data Frames

Q4: Which function is used to reorder rows of a data frame based on a column?

A4: The `arrange()` function from the `dplyr` package is used to reorder rows.

Q5: How do you reshape a data frame from wide to long format?

A5: Use the `pivot_longer()` function from the `tidyr` package to reshape a data frame from wide to long format.

Q6: What function reshapes data from long to wide format?

A6: The `pivot_wider()` function from the `tidyr` package reshapes data from long to wide format.

3. Merging and Joining Data Frames

Q7: What function is used to perform an inner join in R?

A7: The ``inner_join()`` function from the ``dplyr`` package is used to perform an inner join.

Q8: Which function is used for a left join in R?

A8: The ``left_join()`` function from the ``dplyr`` package is used for a left join.

Q9: How do you merge two data frames by a common column?

A9: Use the ``merge()`` function or join functions like ``left_join()`` with a specified by argument, e.g., ``by = "common_column"``.

4. Calculating Summary Statistics

Q10: How do you calculate the mean of a vector in R?

A10: Use the ``mean()`` function to calculate the mean of a vector.

Q11: Which function is used to find the median in R?

A11: The ``median()`` function is used to find the median.

Q12: How can you calculate the standard deviation in R?

A12: Use the ``sd()`` function to calculate the standard deviation.

5. Generating Frequency Tables and Cross-Tabulations

Q13: What function generates a frequency table in R?

A13: The ``table()`` function generates a frequency table.

Q14: How do you create a cross-tabulation in R?

A14: Use the ``xtabs()`` function or ``table()`` to create a cross-tabulation.

6. Understanding Measures of Central Tendency and Dispersion

Q15: What is the measure of central tendency that represents the most frequently occurring value?

A15: Mode represents the most frequently occurring value.

Q16: Which measure of central tendency is sensitive to extreme values?

A16: The mean is sensitive to extreme values.

Q17: What measure of dispersion indicates the average distance from the mean?

A17: Standard deviation indicates the average distance from the mean.

7. Exploring Data Distributions Graphically

Q18: What function can be used to plot a histogram in R?

A18: Use the ``hist()`` function to plot a histogram.

Q19: Which plot is useful for identifying the distribution of a continuous variable?

A19: The bell curve (or density plot) is useful for identifying the distribution of a continuous variable.

Q20: How do you plot a bell curve in R?

A20: Use the ``plot()`` function with ``dnorm()`` to plot a bell curve in R, or use ``ggplot2`` with ``geom_density()``.

MULTIPLE CHOICE QUESTIONS (MCQ'S)

Here are 20 multiple-choice questions (MCQs) with answers related to the points you've provided:

1. Working with Data in R

Q1: Which function is used to create a data frame in R?

a) ``matrix()``

b) ``data.frame()``

- c) ``list()``
- d) ``c()``

Answer: b) ``data.frame()``

Q2: How do you view the structure of a data frame in R?

- a) ``summary()``
- b) ``str()``
- c) ``head()``
- d) ``tail()``

Answer: b) ``str()``

Q3: Which operator is used to access a specific column in a data frame?

- a) ``[]``
- b) ``$``
- c) ``()``
- d) ``<>``

Answer: b) ``$``

2. Reordering and Reshaping Data Frames

Q4: What function in R is used to reorder the rows of a data frame based on a specific column?

- a) ``arrange()``
- b) ``filter()``
- c) ``mutate()``
- d) ``select()``

Answer: a) ``arrange()``

Q5: Which function reshapes data from wide to long format in R?

- a) ``gather()``
- b) ``spread()``
- c) ``pivot_longer()``
- d) ``melt()``

Answer: c) ``pivot_longer()``

Q6: Which function is used to reshape data from long to wide format in R?

- a) ``pivot_wider()``
- b) ``pivot_longer()``
- c) ``spread()``
- d) ``cast()``

Answer: a) ``pivot_wider()``

3. Merging and Joining Data Frames

Q7: Which function is used to perform an inner join in R?

- a) ``left_join()``
- b) ``right_join()``
- c) ``inner_join()``
- d) ``full_join()``

Answer: c) ``inner_join()``

Q8: Which function is used to perform a left join in R?

- a) ``inner_join()``
- b) ``right_join()``

- c) ``full_join()``
- d) ``left_join()``

Answer: d) ``left_join()``

Q9: To merge two data frames by a common column, which function is most appropriate?

- a) ``merge()``
- b) ``join()``
- c) ``combine()``
- d) ``cbind()``

Answer: a) ``merge()``

4. Calculating Summary Statistics

Q10: Which function is used to calculate the mean of a numeric vector in R?

- a) ``mean()``
- b) ``median()``
- c) ``mode()``
- d) ``sum()``

Answer: a) ``mean()``

Q11: What function is used to calculate the median in R?

- a) ``mean()``
- b) ``median()``
- c) ``mode()``
- d) ``range()``

Answer: b) ``median()``

Q12: Which function computes the standard deviation of a numeric vector in R?

- a) ``var()``
- b) ``sd()``
- c) ``mean()``
- d) ``range()``

Answer: b) ``sd()``

5. Generating Frequency Tables and Cross-Tabulations

Q13: Which function generates a frequency table in R?

- a) ``table()``
- b) ``freq()``
- c) ``count()``
- d) ``hist()``

Answer: a) ``table()``

Q14: What function can be used to generate cross-tabulations in R?

- a) ``table()``
- b) ``xtabs()``
- c) ``count()``
- d) ``hist()``

Answer: b) ``xtabs()``

6. Understanding Measures of Central Tendency and Dispersion

Q15: Which measure of central tendency is defined as the middle value in a sorted dataset?

- a) Mean
- b) Median
- c) Mode
- d) Range

Answer: b) Median

Q16: What measure of central tendency is most affected by extreme values (outliers)?

- a) Mean
- b) Median
- c) Mode
- d) Range

Answer: a) Mean

Q17: Which of the following is a measure of dispersion?

- a) Mean
- b) Median
- c) Mode
- d) Standard Deviation

Answer: d) Standard Deviation

7. Exploring Data Distributions Graphically

Q18: Which function is used to create a histogram in R?

- a) ``plot()``
- b) ``hist()``
- c) ``barplot()``
- d) ``pie()``

Answer: b) ``hist()``

Q19: What is a bell curve used to represent in statistics?

- a) The distribution of a categorical variable
- b) The distribution of a normal continuous variable
- c) The distribution of a discrete variable
- d) The distribution of nominal data

Answer: b) The distribution of a normal continuous variable

Q20: How can you plot a density curve in R using ``ggplot2``?

- a) ``geom_density()``
- b) ``geom_histogram()``
- c) ``geom_point()``
- d) ``geom_line()``

Answer: a) ``geom_density()``